

BÀI TẬP LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP)

Python - Bài 5 & 6: Thiết Kế & Xây Dựng Class cho Hệ Thống Thương Mại Điện Tử

PHẦN I: CÂU HỎI LÝ THUYẾT (10 CÂU HỎI)

Câu 1: Phân biệt giữa Class và Object

Class là bản thiết kế quy định các thuộc tính (Attributes) và hành động (Methods) chung cho các Object của nó.

Objects là các sản phẩm thực tế, khi hệ thống chạy, chúng ta khởi tạo (Instantiate) các Object từ Class.

Câu 2: Tác dụng của hàm `__init__()` và tham số `self`

Giải thích tác dụng của hàm `__init__()` trong Python. Tham số `self` là gì và tại sao nó luôn là tham số đầu tiên của mỗi phương thức? Viết ví dụ với class `Product` (sản phẩm).

- Hàm `__init__()` là hàm **khởi tạo trạng thái (initialize)**, nhiệm vụ nhận dữ liệu đầu vào và gán vào các thuộc tính của Object đó.
- `Self` đại diện cho Object đã được khởi tạo và đang thao tác. Vì Class định nghĩa trước Object, nên tham số `self` được dùng để quy định các thuộc tính (Attributes) và hành động (Methods) đến khi khởi tạo Object hệ thống sẽ thay tên của Object vào chỗ tham số `self` và thực thi code.



```
class Product:
    # 1. Hàm __init__ là trạm lắp ráp.
    # self đứng đầu để nhận diện object nào đang được tạo.
    def __init__(self, product_id: int, name: str, price: float):
        self.id = product_id      # Gắn id vào object (self)
        self.name = name          # Gắn name
        self.price = price        # Gắn price
        self.stock = 0            # Giá trị mặc định

    def update_stock(self, quantity: int):
        # 2. Nhờ có self, hàm này biết cần update tồn kho cho SẢN PHẨM NÀO
        self.stock += quantity
        print(f"[{self.name}] Stock updated to: {self.stock}")

# Quá trình khởi tạo (Instantiation)
# Khi chạy dòng này, Python ngằm ném 'macbook' vào vị trí của 'self' trong __init__
macbook = Product(1, "MacBook Pro", 2000.0)

# Khi gọi method, Python ngằm dịch thành: Product.update_stock(macbook, 10)
macbook.update_stock(10)
```

... [MacBook Pro] Stock updated to: 10

Câu 3: Các loại thuộc tính và phương thức trong Class

Liệt kê và giải thích ba loại phương thức trong Python: Instance Method, Class Method(@classmethod), Static Method (@staticmethod). Cho ví dụ cụ thể cho mỗi loại trong bối cảnh class Order (đơn hàng).

Core Concept (The Mental Model)

- **Instance Method (Phương thức của Đối tượng):**
 - Luôn nhận `self` làm tham số đầu tiên.
 - **Mental Model:** "Tôi thao tác trên data của MỘT ĐƠN HÀNG cụ thể".
- **Class Method (Phương thức của Lớp):**
 - Sử dụng decorator `@classmethod`. Luôn nhận `cls` (đại diện cho chính Class đó) làm tham số đầu tiên.
 - **Mental Model:** "Tôi thao tác trên data cấu hình CHUNG CỦA MỌI ĐƠN HÀNG" hoặc "Tôi là một cỗ máy sản xuất ra các đơn hàng (Factory)".
- **Static Method (Phương thức Tĩnh):**
 - Sử dụng decorator `@staticmethod`. KHÔNG nhận `self` cũng KHÔNG nhận `cls`.
 - **Mental Model:** "Tôi chỉ là một hàm tiện ích (utility). Tôi được nhét vào class Order chỉ vì tôi có liên quan mật logic tới Order, giúp code gọn gàng hơn thay vì vứt tôi vạ vật bên ngoài".

Tưởng tượng Class `Order` (Đơn hàng) giống như một **Cửa hàng đồ ăn nhanh**.

- **Instance Method (Hành động của Khách):** Khách hàng A gọi một hóa đơn, khách B gọi một hóa đơn. Việc "Tính tổng tiền" của hóa đơn A chỉ áp dụng cho đồ ăn của A. Nó cần dữ liệu cụ thể của từng hóa đơn.

- **Class Method (Hành động của Quản lý):** Quản lý cửa hàng nhận một tờ giấy fax từ chi nhánh khác chứa thông tin đặt hàng, và từ tờ giấy đó đúc ra một hóa đơn mới. Hành động này là "tạo ra hóa đơn" (Factory), chứ không phải thao tác trên một hóa đơn có sẵn.
- **Static Method (Cái máy soi tiền giả):** Cửa hàng có một cái máy soi mã vạch/tiền giả. Bất kỳ ai cũng có thể nhét tờ tiền vào để kiểm tra đúng/sai. Cái máy này không cần biết hóa đơn của ai, cũng không cần quyền quản lý cửa hàng. Nó chỉ là công cụ tiện ích.

Câu 4: Tính đóng gói (Encapsulation) và access modifiers

Python không có access modifiers như Public, Protected, Private nhưng sử dụng convention với dấu gạch dưới (_). Hãy giải thích ba mức độ: public (không có dấu), protected (_name), private (__name). Giải thích lý do sử dụng getter/setter với ví dụ class User có thuộc tính __password.

Python dùng **Convention (Quy ước đặt tên)** để "nhắc nhở":

1. Public (name - Không gạch dưới):

- *Ý nghĩa:* "Dùng thoải mái đi, tôi thiết kế cái này cho anh dùng đấy."

2. Protected (_name - 1 gạch dưới):

- *Ý nghĩa:* "Cảnh báo! Đây là dữ liệu dùng nội bộ trong Class này và các Class con (Inheritance) của nó. Anh gọi từ bên ngoài vào vẫn chạy được, nhưng nếu tương lai code lỗi thì tự chịu trách nhiệm nhé."

3. Private (__name - 2 gạch dưới):

- *Ý nghĩa:* "Cấm đụng vào! Tôi là dữ liệu nhạy cảm."
 - *Cơ chế:* Python sẽ làm một trò ảo thuật gọi là **Name Mangling** (Băm tên). Khi nó thấy __name trong class **User**, nó sẽ tự động đổi tên biến thành **_User__name**. Nhờ vậy, code bên ngoài gọi **user.__name** sẽ bị báo lỗi không tìm thấy.
 - **Lầm tưởng __ là tuyệt đối an toàn:** Thực ra, trong Python, bạn vẫn có thể hack vào bằng cách gọi **user._User__password**. Tính năng Name Mangling chỉ là "cái khóa cổng", dùng để chống *nhầm lẫn vô ý* khi code, chứ không chống được *cố ý phá hoại* hoặc bảo mật cấp hệ thống.
- **Getter (Accessors):** Là phương thức dùng để **đọc** giá trị của một thuộc tính private. Nó cho phép bạn kiểm soát *ai* được đọc và *đọc* dưới *định dạng* nào.

- **Setter (Mutators):** Là phương thức dùng để **ghi/cập nhật** giá trị của một thuộc tính private. Nó là chốt chặn (choke-point) để kiểm tra tính hợp lệ của dữ liệu trước khi thực sự lưu vào vùng nhớ.

Mental Model: Đừng coi dữ liệu là một "cái hộp rỗng" ai ném gì vào cũng được. Hãy coi nó là một "hộp đồng" – bạn phải tuân thủ các điều khoản (logic kiểm tra) thì mới được phép thay đổi nó.

Tưởng tượng hệ thống phần mềm giống như một **Ngân hàng**.

- Thuộc tính `__password` chính là **Két sắt** chứa tiền.
- Việc cho phép truy cập trực tiếp (`user.password = "123"`) giống như việc ngân hàng mở toang cửa két, để khách hàng tự đi vào, tự bỏ tiền vào hoặc rút tiền ra. Điều này cực kỳ rủi ro: Khách có thể vô tình làm rách tiền, bỏ nhầm tiền giả, hoặc lấy cắp.

Giải pháp: Ngân hàng thuê một **Giao dịch viên (Teller)**.

- Bạn muốn gửi tiền? Đưa cho giao dịch viên (Setter). Họ sẽ soi xem tiền thật hay giả (Validation), ghi chép sổ sách rồi mới cất vào két.
- Bạn muốn xem số dư? Hỏi giao dịch viên (Getter). Họ kiểm tra CMND của bạn rồi mới đọc số dư.

Getter/Setter sinh ra làm **người gác cổng (Gatekeeper)** bảo vệ dữ liệu nội bộ của Object.

Câu 5: Kế thừa (Inheritance) và ghi đè phương thức

Giải thích khái niệm kế thừa trong OOP. Làm thế nào để ghi đè (override) một phương thức từ lớp cha? Viết ví dụ: tạo lớp cha Person (người), sau đó tạo lớp con Customer (khách hàng) kế thừa từ Person với phương thức khác nhau.

Core Concept (The Mental Model)

- **Kế thừa (Inheritance):** Thể hiện mối quan hệ **"IS-A" (Là một)**. Khách hàng *là một* Người. Admin *là một* Người. Lớp con (Child Class) sẽ thừa hưởng toàn bộ tài sản (Attributes) và kỹ năng (Methods) của Lớp cha (Parent Class).
- **Ghi đè (Overriding):** Là hành động lớp con nói với lớp cha: *"Cảm ơn vì đã cho con kỹ năng này, nhưng con muốn thực hiện nó theo cách của riêng con."* Lớp con tạo ra một Method trùng tên chính xác với Method của cha, nhưng thay đổi logic bên trong.

```
# --- LỚP CHA ---
class Person:
    def __init__(self, name: str, email: str):
        self.name = name
        self.email = email

    # Phương thức chung
    def login(self):
        print(f"[System] {self.email} đang đăng nhập...")

    # Phương thức sẽ bị ghi đè
    def get_role_description(self) -> str:
        return "Tôi là một người dùng hệ thống bình thường."
```

```
# --- LỚP CON ---
class Customer(Person):
    def __init__(self, name: str, email: str, customer_id: str):
        # 1. Gọi __init__ của cha để gán name và email (Không tự gán lại)
        super().__init__(name, email)

        # 2. Khởi tạo thuộc tính riêng của lớp con
        self.customer_id = customer_id
        self.cart = []

    # 3. GHI ĐÈ PHƯƠNG THỨC TỪ CHA (Overriding)
    def get_role_description(self) -> str:
        # Tự định nghĩa lại logic, hoặc có thể lấy kết quả từ cha để chế biến thêm
        return f"Tôi là Khách hàng VIP. Mã KH: {self.customer_id}."

    # 4. PHƯƠNG THỨC KHÁC BIỆT (Chỉ Customer mới có)
    def add_to_cart(self, item_name: str):
        self.cart.append(item_name)
        print(f"[Shop] '{item_name}' đã được thêm vào giỏ hàng của {self.name}.")
```


Câu 6: Scope (Phạm vi hoạt động) của biến trong class

Giải thích khái niệm Scope (phạm vi) trong Python. Phân biệt giữa: biến global (toàn cục), biến local (cục bộ trong hàm), biến instance (thuộc về object), và biến class (thuộc về class). Cho ví dụ cụ thể cho mỗi loại trong class ShoppingCart.

Tưởng tượng phần mềm của bạn là một **Trung tâm thương mại (Mall)**.

- **Global (Toàn cục):** Địa chỉ của trung tâm thương mại. Bất kỳ ai (từ bảo vệ, khách hàng, đến nhân viên) đều biết địa chỉ này. Nó tồn tại từ lúc khai trương đến lúc phá dỡ.
- **Class (Thuộc về Lớp):** Chính sách của riêng siêu thị Điện Máy trong Mall (ví dụ: "Tất cả màn hình đều bảo hành 2 năm"). Nó áp dụng cho mọi chiếc TV bán ra ở siêu thị này.
- **Instance (Thuộc về Đối tượng):** Giỏ hàng riêng của một người khách. Giỏ hàng của tôi có thịt, giỏ hàng của bạn có rau. Chúng không liên quan gì đến nhau.
- **Local (Cục bộ):** Tờ giấy nháp người thu ngân dùng để tính nhằm tiền thối lại cho bạn. Tính xong, tờ giấy bị vứt vào thùng rác ngay lập tức.

Vấn đề Scope giải quyết: Quản lý vòng đời bộ nhớ (Memory Lifecycle) và Cách ly dữ liệu (Data Isolation). Nếu mọi thứ đều là Global, RAM của bạn sẽ đầy ngay lập tức và giỏ hàng của người này sẽ bị lẫn đồ của người khác.

Scope trong Python quyết định 2 thứ:

1. **Visibility (Ai nhìn thấy):** Đoạn code ở dòng X có thể đọc/ghi biến Y hay không.
2. **Lifetime (Sống bao lâu):** Khi nào thì Garbage Collector (GC - Bộ dọn rác) được phép xóa biến đó khỏi RAM.

Trong hệ thống E-commerce, class `ShoppingCart`:

- **Global (`STORE_OPEN_STATUS`):** Backend có một biến global cấu hình trạng thái đóng/mở cửa của toàn hệ thống (bảo trì). Mọi class đều cần đọc nó trước khi cho phép mua hàng.
- **Class (`max_items_allowed`):** Quy định chung của hệ thống: "Mỗi giỏ hàng chỉ được chứa tối đa 50 món để chống spam". Quy định này là chung cho MỌI giỏ hàng.
- **Instance (`self.items`):** Danh sách các món hàng thực tế mà người dùng Nguyễn Văn A đã thêm vào. User B sẽ có một `self.items` khác trên vùng nhớ khác.
- **Local (`discounted_price`):** Nằm trong hàm `calculate_total()`. Nó chỉ là một biến tạm dùng để lưu kết quả trung gian lúc tính toán, tính xong trả về kết quả là nó biến mất.

```
# 1. BIẾN GLOBAL (Toàn cục)
# Thường viết hoa để phân biệt. Tồn tại suốt vòng đời app.
PLATFORM_FEE = 5000

class ShoppingCart:
    # 2. BIẾN CLASS (Class Variable)
    # Thuộc về bản thân class ShoppingCart.
    # Mọi giỏ hàng đều chia sẻ chung biến này.
    max_items_allowed = 50
    total_carts_created = 0 # Dùng để tracking số lượng giỏ hàng đã tạo

    def __init__(self, customer_id: str):
        # 3. BIẾN INSTANCE (Instance Variable)
        # Bắt đầu bằng chữ 'self.'. Đây là tài sản RIÊNG của mỗi object.
        self.customer_id = customer_id
        self.items = []

        # Cập nhật biến class (cần gọi qua tên Class, không gọi qua self)
        ShoppingCart.total_carts_created += 1

    def add_item(self, item_name: str, price: int):
        # Truy cập biến class để check logic chung
        if len(self.items) >= ShoppingCart.max_items_allowed:
            raise ValueError("Giỏ hàng đã đầy!")

        self.items.append({"name": item_name, "price": price})
```

```

def calculate_total(self) -> int:
    # 4. BIẾN LOCAL (Biến cục bộ)
    # Biến 'subtotal' chỉ tồn tại trong quá trình hàm này chạy.
    # Hàm chạy xong (chạm đến lệnh return), 'subtotal' bị hủy khỏi RAM.
    subtotal = 0

    for item in self.items:
        subtotal += item["price"]

    # Sử dụng biến Global + biến Local
    return subtotal + PLATFORM_FEE

# --- THỬ NGHIỆM THỰC TẾ ---

cart_alice = ShoppingCart("Alice_001")
cart_bob = ShoppingCart("Bob_002")

# Instance Scope hoạt động: Giỏ của ai người nấy giữ
cart_alice.add_item("iPhone", 1000)
print(cart_alice.items) # [{'name': 'iPhone', 'price': 1000}]
print(cart_bob.items)   # [] (Của Bob vẫn trống)

# Class Scope hoạt động: Chia sẻ chung
print(ShoppingCart.total_carts_created) # Kết quả: 2

# Nếu Admin thay đổi chính sách hệ thống (Biến Class)
ShoppingCart.max_items_allowed = 100
# Lập tức quy định mới áp dụng cho cả Alice, Bob và mọi khách hàng tương lai.

```

Câu 7: Nguyên lý POLYMORPHISM và ABSTRACTION của OOP

- POLYMORPHISM (Tính đa hình): Cùng một interface (class) nhưng nhiều cách triển khai khác nhau. Ưu điểm? Cho ví dụ: method `move()` được override trong Dog, Cat, Bird nhưng có kết quả khác nhau.

- ABSTRACTION (Tính trừu tượng): Ẩn đi các chi tiết phức tạp, chỉ cung cấp interface đơn giản cho người dùng. Khác gì với Encapsulation? Cho ví dụ: class Shape với abstract method `area()`.

A. POLYMORPHISM (Tính Đa Hình)

- **Bản chất:** "Cùng một nút bấm, nhưng hành động khác nhau tùy thuộc vào kẻ thực thi."
- **Mental Model:** Hãy nghĩ đến lệnh `move()` (di chuyển). Nếu bạn ra lệnh `move()` cho một con Chó, nó sẽ **chạy**. Ra lệnh cho con Chim, nó sẽ **bay**. Ra lệnh cho con Mèo, nó sẽ **nhảy**. Khái niệm "di chuyển" là chung (interface chung), nhưng cách thực hiện là đa dạng (đa hình).

B. ABSTRACTION (Tính Trừu Tượng)

- **Bản chất:** "Che giấu sự phức tạp, chỉ đưa ra một Giao diện (Interface) đơn giản để sử dụng."
- **Mental Model:** Chiếc vô lăng ô tô. Bạn chỉ cần biết xoay trái thì xe sang trái. Bạn **KHÔNG CẦN BIẾT** hệ thống trợ lực điện, bánh răng, thước lái bên dưới hoạt động phức tạp ra sao. Vô lăng chính là Abstraction.

SỰ KHÁC BIỆT: ABSTRACTION vs ENCAPSULATION

Đây là câu hỏi Interview kinh điển nhất:

- **Encapsulation (Đóng gói):** Giấu đi **Dữ liệu (State)**. Giống như bỏ tiền vào két sắt khóa lại để không ai ăn cắp được (dùng private, getter/setter). Trọng tâm là **Bảo mật / Toàn vẹn dữ liệu**.

- **Abstraction (Trừu tượng):** Giấu đi **Độ phức tạp (Complexity/Implementation)**. Giống như cái cây ATM, giấu đi hệ thống server ngân hàng phức tạp, chỉ cho bạn cái màn hình để bấm nút rút tiền. Trọng tâm là **Thiết kế giao diện dễ dùng**.

Trong một hệ thống E-commerce (ví dụ gửi thông báo cho User):

- **Abstraction:** Kỹ sư tạo ra một Abstract Class **Notifier**. Trong đó định nghĩa một abstract method là **send()**. Nó quy định: *"Bất kỳ module thông báo nào cũng BẮT BUỘC phải có hàm **send()**, còn send đi đâu tôi không quan tâm."*
- **Polymorphism:** Tạo ra 3 class con: **EmailNotifier**, **SMSNotifier**, **PushNotifier**. Cả 3 đều triển khai hàm **send()**.
 - **EmailNotifier.send()**: Mở kết nối SMTP, gắn file đính kèm, gửi.
 - **SMSNotifier.send()**: Gọi API của Twilio, tốn 500đ/tin nhắn.
- **Data Flow:** Ở Controller, bạn nhận được một mảng chứa các thiết lập của User: **notifiers = [EmailNotifier(), SMSNotifier()]**. Hệ thống chỉ cần chạy vòng lặp gọi **notifier.send()**. Đa hình sẽ tự động kích hoạt logic tương ứng.

Chúng ta tạo ra một "bản hợp đồng" (Abstraction) tên là **Shape**, yêu cầu mọi hình khối bắt buộc phải tự biết cách tính diện tích của nó qua hàm **area()**. Khi cần tính tổng, hệ thống chỉ việc nhắm mắt gọi **item.area()**, tự động hình tròn sẽ dùng công thức hình tròn, hình vuông sẽ dùng công thức hình vuông (Polymorphism).

```

from abc import ABC, abstractmethod
import math

# 1. ABSTRACTION: Định nghĩa Interface / Hợp đồng
class Shape(ABC):
    # Dùng decorator này để Ép BUỘC các class con phải viết lại hàm area()
    # Nếu class con quên viết, chương trình sẽ crash ngay lúc khởi tạo.
    @abstractmethod
    def area(self) -> float:
        """Trả về diện tích của hình khối"""
        pass

# 2. IMPLEMENTATION: Triển khai các hình cụ thể
class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    # Bắt buộc phải override hàm area()
    def area(self) -> float:
        return math.pi * (self.radius ** 2)

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    # Bắt buộc phải override hàm area()
    def area(self) -> float:
        return self.width * self.height

```

Câu 8: Schema (Sơ đồ thiết kế class) - Cách lập kế hoạch xây dựng class

Giải thích khái niệm Schema trong thiết kế OOP. Các bước cần làm trước khi viết code:

- Xác định các thuộc tính (attributes) và phương thức (methods) cần thiết
- Xác định mối quan hệ giữa các class (kế thừa, composition)
- Xác định public/private cho mỗi thành phần

Hãy vẽ/mô tả schema cho class `PaymentProcessor` (xử lý thanh toán) với các class con `CreditCardPayment`, `BankTransferPayment`, `E-WalletPayment`.

Tưởng tượng bạn đang xây dựng một **Tòa nhà Landmark**. Bạn không thể mua gạch về và nói với thợ xây: *"Cứ xây đi, lát nữa xem phòng khách nằm đâu rồi tính tiếp"*. Bạn cần một **Bản thiết kế kiến trúc (Blueprint)**.

Trong OOP, **Schema (Sơ đồ thiết kế class)** chính là bản blueprint đó. Nó giải quyết bài toán:

- Tránh hiện tượng "Code Spagetti" (Class này gọi class kia chẳng chịt, rối rắm).
- Giúp cả team hiểu chung một luồng logic trước khi chia task (Người làm class A biết chắc chắn class B sẽ có những hàm gì để gọi).

2. Core Concept (The Mental Model)

Schema (thường được vẽ dưới dạng UML Class Diagram) là quá trình bạn trả lời 3 câu hỏi lớn trước khi code:

1. Xác định Noun (Danh từ) & Verb (Động từ):

- *Danh từ* biến thành **Class** và **Attributes (Thuộc tính)**. (Ví dụ: "Hóa đơn có Số tiền và Ngày tạo" -> Class `Invoice`, attributes `amount`, `created_at`).

- *Động từ* biến thành **Methods (Phương thức)**. (Ví dụ: "Khách hàng thanh toán hóa đơn" -> Method `pay()`).

2. Xác định Access Modifiers (Quyền truy cập):

- Cái gì là **Public (+)**: Cửa chính để thế giới bên ngoài giao tiếp.
- Cái gì là **Private (-)**: Bí mật kinh doanh, logic nội bộ cần giấu kín (Encapsulation).
- Cái gì là **Protected (#)**: Chia sẻ cho con cái (Inheritance) nhưng giấu với người ngoài.

3. Xác định Relationship (Mối quan hệ):

- **Inheritance (Kế thừa - IS-A)**: `CreditCard` là một loại `Payment`.
- **Composition (Lắp ráp - HAS-A)**: `Order` chứa một `Payment`.

```

=====
[ABSTRACT CLASS] PaymentProcessor  (Ổ cắm điện)
=====
- transaction_id: String (Private)
- amount: Float (Private)
- status: String (Private - PENDING/SUCCESS/FAIL)
-----
+ get_status(): String (Public)
# log_audit(msg: String) (Protected - Dùng chung)
+ process_payment(): Boolean [ABSTRACT] (Bắt buộc class con phải tự làm)
=====
      ^
      | (Kế thừa / Ký hợp đồng)
      |
-----
|           |           |
[CreditCard] [BankTransfer] [EWallet]
-----
- card_number      - bank_code      - wallet_provider
- cvv              - account_number - phone_number
- expiry_date
-----
+ process_payment() + process_payment()

```


Câu 9: Cách vẽ sơ đồ tư duy (Mind Map) để thiết kế class

Giải thích lợi ích của việc vẽ sơ đồ tư duy trước khi code. Các thành phần chính trong sơ đồ tư duy:

- Tên class ở trung tâm
- Các nhánh chính: Thuộc tính, Phương thức, Quan hệ với class khác
- Chi tiết: tên, kiểu dữ liệu, access level

Hãy mô tả sơ đồ tư duy cho class InventoryManager (quản lý kho hàng).

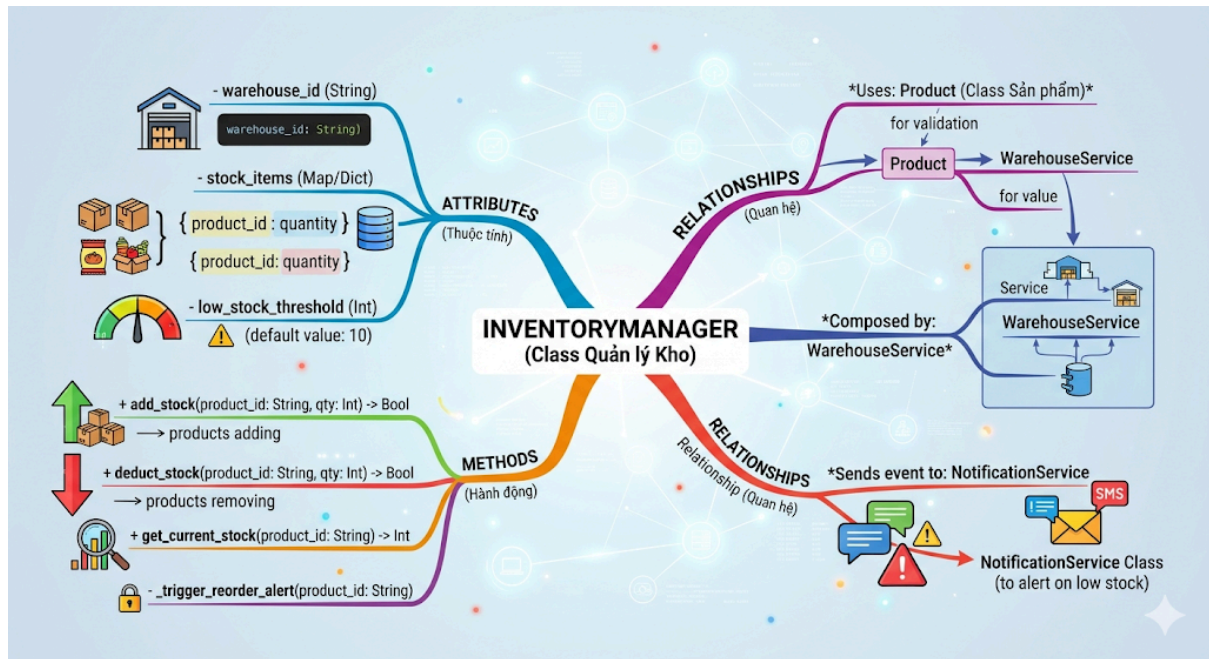
Vấn đề Mind Map giải quyết: Tách biệt hoàn toàn quá trình **Thiết kế (Design)** và quá trình **Thực thi (Implementation)**. Nó cho phép bạn nhìn thấy "bức tranh toàn cảnh" và cấu trúc DNA của một Class chỉ trong một ánh nhìn. Áp dụng tư duy 80/20: Bạn chỉ cần bỏ ra 20% thời gian đầu dự án để vẽ Mind Map, nó sẽ giúp bạn loại bỏ 80% các lỗi logic và đập đi xây lại khi vào giai đoạn code.

Mind Map cho một Class không phải là một tài liệu kỹ thuật khô khan, nó là một cái cây sinh động tỏa ra từ trung tâm.

Một sơ đồ chuẩn mực gồm 4 thành phần:

1. **Trung tâm (Center):** Tên của Class (Danh từ).
2. **Nhánh 1 - Attributes (Thuộc tính):** Biểu diễn trạng thái, dữ liệu mà Class này nắm giữ (Ví dụ: số lượng, tên, ID).
3. **Nhánh 2 - Methods (Phương thức):** Biểu diễn hành vi, những gì Class này có thể làm (Ví dụ: thêm, xóa, cập nhật).
4. **Nhánh 3 - Relationships (Mối quan hệ):** Class này tương tác với ai? Nhận dữ liệu từ đâu? Kế thừa từ class nào?

Lưu ý: Trên mỗi nhánh con, bạn phải ký hiệu rõ: **Tên**, **Kiểu dữ liệu** (String, Int, List), và **Access Level** (+ cho Public, - cho Private).



Mô tả Sơ đồ tư duy (Mind Map) cho class **InventoryManager**:

- **TRUNG TÂM: InventoryManager** (Quản lý Kho)
 - **NHÁNH 1: Attributes** (Dữ liệu nội bộ)
 - - **warehouse_id** (String): ID của nhà kho (Ví dụ: "KHO_CHB_01").
 - - **stock_items** (Map/Dict): Lưu trữ số lượng hiện tại. Dạng {product_id: quantity}.
 - - **low_stock_threshold** (Int): Ngưỡng cảnh báo hết hàng (Mặc định = 10).
 - **NHÁNH 2: Methods** (Hành động)
 - + **add_stock**(product_id: String, qty: Int) -> Bool: Nhập hàng vào kho.
 - + **deduct_stock**(product_id: String, qty: Int) -> Bool: Xuất hàng (chỉ thành công nếu tồn kho >= qty).
 - + **get_current_stock**(product_id: String) -> Int: Xem tồn kho.
 - - **_trigger_reorder_alert**(product_id: String): Hàm chạy ngầm, tự động báo mua thêm khi tồn kho tụt dưới ngưỡng.

-  **NHÁNH 3: Relationships (Quan hệ)**
 - *Uses:* **Product** (Class Sản phẩm) - Dùng để xác thực product_id có tồn tại không.
 - *Composed by:* **WarehouseService** - Nằm bên trong service quản lý tổng.
 - *Sends event to:* **NotificationService** - Để gửi tin nhắn cho quản lý khi sắp hết nguyên liệu.